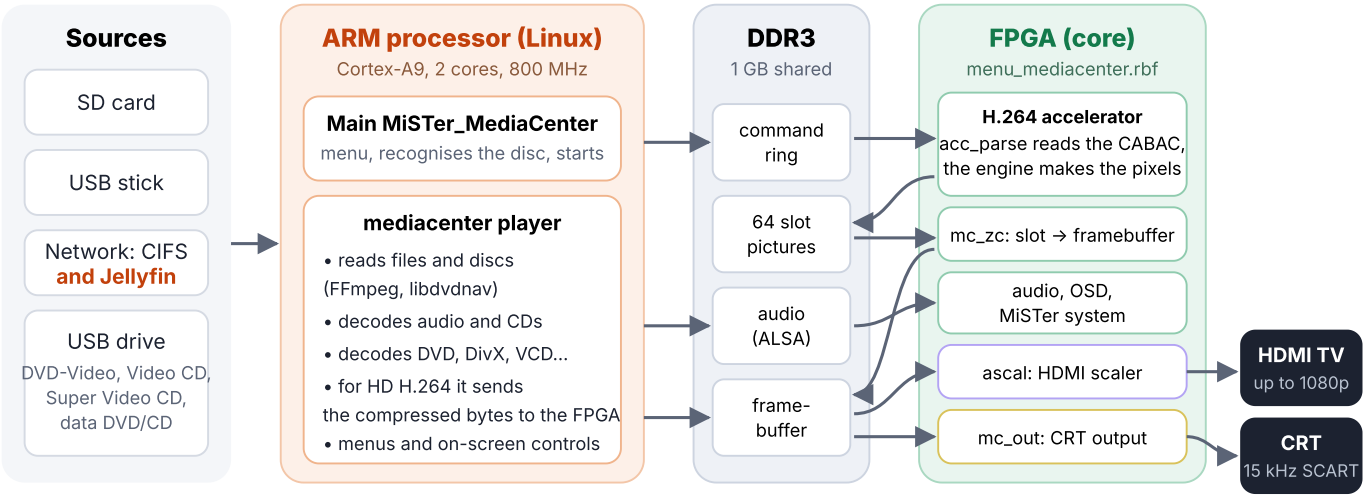
COMPLETE DOCUMENTATION

How MiSTer MediaCenter 4.5 works, from the disc to the screen

The FPGA chip, the player on the ARM processor and how they hand work to each other, the journey of every picture to HDMI and CRT, what is new in 4.5 and the recipe to rebuild everything from scratch.



The system in one picture



ARM and FPGA never talk directly: they leave data and messages for each other in fixed areas of the DDR3 memory.

Version 4.5 at a glance

Release	**v4.5**, 12 November 2026, commit `307a7c5`
ARM player	`mediacenter` build **224**
FPGA core	`menu_mediacenter.rbf`, seed **12** MD5 `7cb8175f995d5b72a49c2ef032fbc963` the same file as in 4.0
Board	Terasic DE10-Nano, chip Intel Cyclone V SE `5CSEBA6U23I7`
What's new	Jellyfin, audio screen, audio CDs, Opus and tracker modules, Main of 11 Nov, new script name

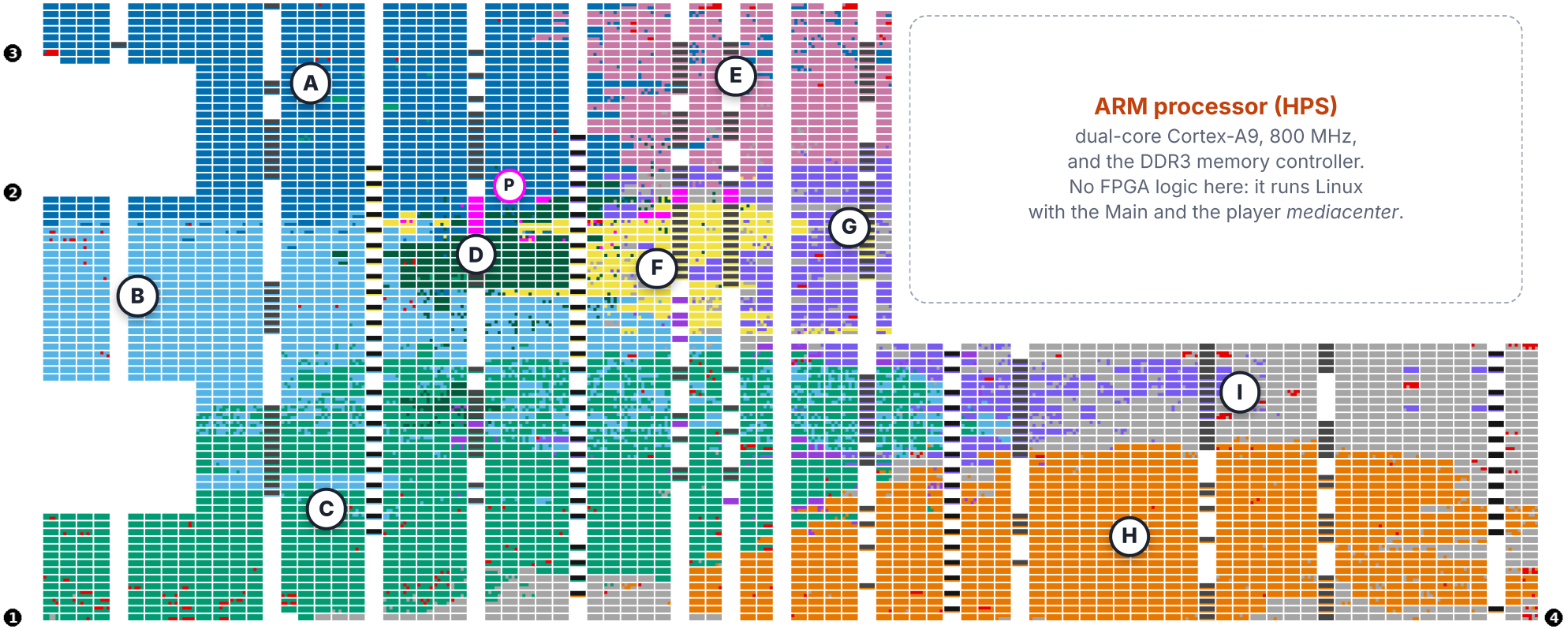
The pages

2	The chip map (landscape page)
3	What's new in 4.5 and the ARM–FPGA links
4–5	Chip resources, clocks and blocks
6	Journey 1: from the source to the player
7	Journey 2: who decodes what
8	Journey 3: the ARM path and the sound
9	Journey 4: the FPGA path
10	Shared memory and messages
11	HDMI and CRT outputs, the logs
12–13	Journey 5: Jellyfin
14	The audio screen and the equalizer
15	Audio CDs, formats, Main, script
16–17	Rebuilding everything from scratch
18–19	Glossary, source map

How to read the diagrams in this document. The same colours are used on every page: light orange = ARM processor (programs), grey-blue = shared DDR3 memory, green = FPGA (circuits). The pills 108 100 85.7 27 give the clock of a block in MHz. Arrows follow the data. The bottom of each page names the repository files the information comes from.

2 The FPGA chip seen from above: seed 12 core (4.0 and 4.5)

Drawn from the real position of the 120,318 cells in the Quartus 17.0.2 database



A	Deblocking filter (acc_dbk)	100	6,628 ALM · 15.8%
B	Motion compensation	100	6,094 · 14.6%
C	Engine: IDCT, intra, reconstr.	100	8,886 · 21.2%
D	Queue and A6 switch (mc_acc)	100	1,033 · 2.5%
E	Copy into the framebuffer (mc_zc)	108	2,506 · 6.0%
F	15 kHz CRT output (mc_out)	27	1,022 · 2.4%
G	HDMI scaler (ascal)	HDMI	2,270 · 5.4%
H	acc_parse: CABAC reader	85.7	7,278 · 17.4%
I	MiSTer system, rest of core		5,594 · 13.4%
P	Bridges 108↔100 and 100↔85.7		48 + 166
	Completely empty ALMs		345 · 0.8%
	Dark rows: M10K memories · black: DSP		

How to read it

Each small square is an **ALM**, the FPGA's logic brick: it does a small calculation and remembers a few bits. The chip has 41,910 of them, in groups of 10 (the 5×2 rectangles, called LABs). The colour says which part of the core the ALM belongs to. The coloured pills are the clocks: 100 = 100 MHz and so on.

The clock generators (black numbers at the edges)

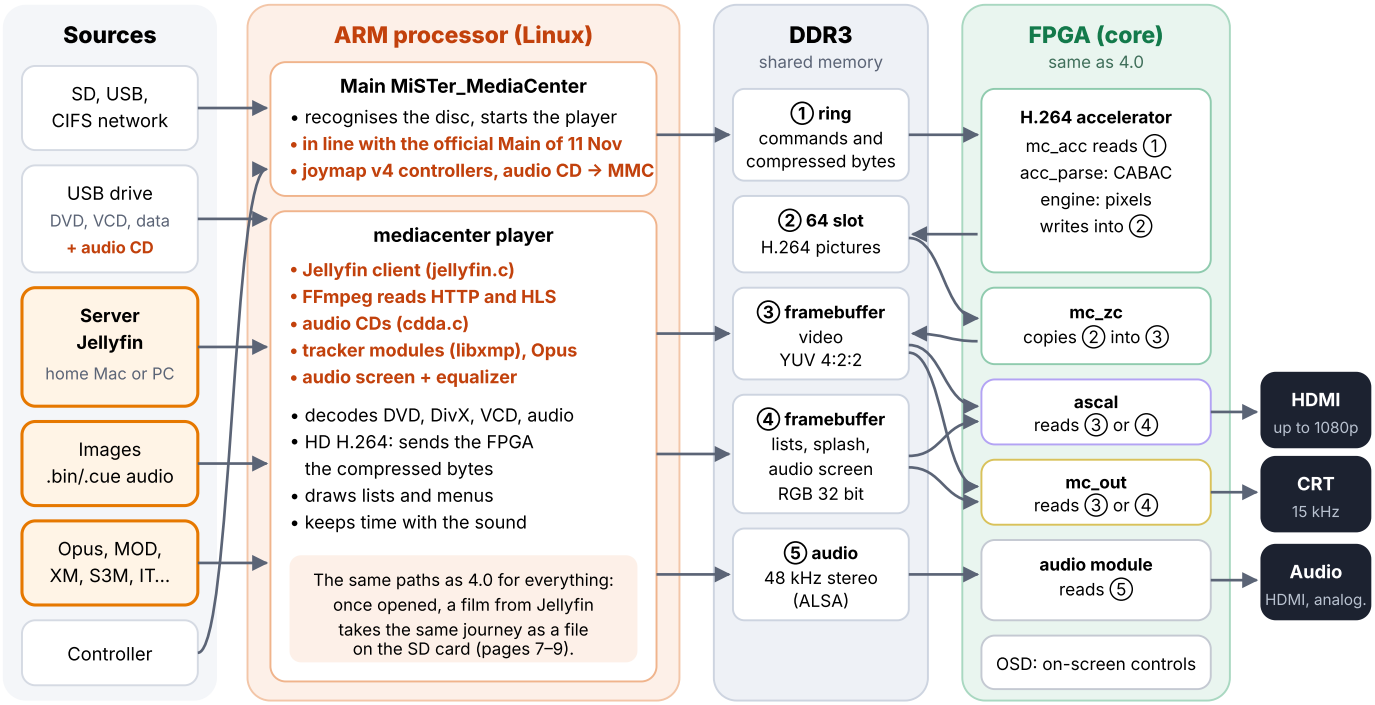
❶ HDMI PLL, reprogrammed at each resolution change. ❷ Accelerator PLL: 100 MHz to the engine, 85.7 MHz to acc_parse. It is fixed far from ❶: when they were close HDMI went out of range at 720p. ❸ Audio PLL, 24.576 MHz. ❹ Core PLL: 108 and 27 MHz.

What it tells us

The chip is full: only 345 empty ALMs are left, scattered. The legend numbers count every ALM touched even partly (41,525). Quartus counts 39,305 ALMs needed (94%): the difference is ALMs only half used. Parts that talk a lot sit close together: the engine on the left, the video outputs in the middle near the ARM, acc_parse in a compact block bottom right.

3 What's new in 4.5 and how ARM and FPGA work together

Version 4.5 changes only the programs that run on the ARM processor: the player `mediacenter` and the Main. The FPGA core is the same file as in 4.0, byte for byte. The new features use the paths the FPGA already offered.



Legend. Dark orange marks what is new in 4.5; boxes with an orange border are new sources. Numbers ①–⑤ are the DDR3 memory areas through which ARM and FPGA hand work to each other: they never talk directly.

What's new	Where it runs	What it uses in the FPGA	Page
Jellyfin client	player, `jellyfin.c`	H.264 accelerator (original files and server conversions), HDMI and CRT outputs	12–13
Audio screen with equalizer	player, `audio_ui.c`	RGB framebuffer ④, ascal and mc_out, audio module	14
Audio CDs inside MMC	Main (recognises) + player, `cdda.c`	audio module, audio screen	15
Opus and tracker modules	player: FFmpeg and libxmp, `modplay.c`	audio module, audio screen	15
Main in line with the official one	Main	nothing new: it loads the same core	15
New script name and picture	script, player (`splash.h`)	framebuffer ④	15

The four paths between ARM and FPGA, in short

1 · Video decoded by the ARM (DVD, DivX, Video CD...): the ARM makes the pixels and copies them into ③; the FPGA converts the colours and fits the size to HDMI and CRT.

2 · HD H.264 video with the accelerator (also from Jellyfin): the ARM writes the compressed bytes into ①; the FPGA rebuilds the pictures in ②, `mc_zc` copies them into ③ at the right time.

3 · Drawn screens (lists, Jellyfin menus, splash, audio screen): the ARM draws into ④ at 32 bits; the FPGA shows them; the core blends in the playback controls (OSD).

4 · Sound: the ARM decodes, applies the equalizer to music, and sends the samples to ALSA (⑤); the core's audio module takes them to HDMI, analogue output and S/PDIF.

4 How much room each part uses

The FPGA has four kinds of resources: logic is almost used up, memories and multipliers still have room. The 4.5 core is the same as the 4.0 one, so these numbers hold for both.

Logic (ALM)	M10K memories	DSP multipliers	Pins
94%	**43%**	**74%**	**46%**
39,305 of 41,910	238 of 553 blocks of 10 kbit	83 of 112	145 of 314

ALM: the logic brick, does small calculations and remembers 2–4 bits. **Registers**: the memory bits inside the ALMs. **M10K**: internal 10 kbit memories, for tables and picture lines. **DSP**: fast multipliers, for filters and pixel maths. Official Quartus figures (report *Fitter Resource Utilization by Entity*); a module includes its sub-modules. Adding up the modules, which also use fractions of ALMs, the rounded total is 39,306.

Part of the core (file)	Clock (MHz)	ALM	Registers	M10K	DSP
Whole chip (sys_top)		**39,306**	**54,398**	**238**	**83**
MediaCenter core (menu.sv)		32,804	44,111	175	53
H.264 accelerator (mc_acc.sv and sub-modules)	100	28,818	38,964	118	33
Command queue, A6 switch, tables	100	951	1,823	7	
Bridges 108 ↔ 100 MHz (mc_afifo)	108 100	44	92	9	
Engine (acc_engine.sv): intra, commands, writing	100	5,005	7,701	18	3
Deblocking filter (acc_dbk.sv)	100	6,417	11,474	26	
Motion compensation (acc_mc.sv)	100	6,339	6,848	11	21
Inverse transform (acc_idct.sv)	100	1,627	3,052		
Filter strength (acc_bs.sv)	100	913	582	8	
Residuals and reconstruction (acc_resid, acc_recon)	100	448	273	12	
CABAC reader acc_parse + cab_dec	85.7	6,872	6,837	28	9
4 bridges 100 ↔ 85.7 MHz (mc_afifo)	100 85.7	103	227	7	
Copy into the framebuffer (mc_zc.sv)	108	2,299	2,942	29	
CRT output (mc_out.sv)	27 108	970	1,066	28	20
Link with the ARM (hps_io), bus, SDRAM	108	705	1,139		
MiSTer framework (sys/ folder)		6,501	10,287	63	30
HDMI scaler (ascal.vhd)	HDMI 108	2,034	3,775	42	22
Audio (audio_out, alsa)	24.6	1,189	1,601		8
On-screen menu OSD (osd.v ×2)	video	1,008	1,298	16	
HDMI PLL control, memory, outputs, rest	various	2,270	3,613	5	

ALMs per part (of 39,306)



- Engine (with filter and motion) 20,800 · 53%
- CABAC reader acc_parse 6,872 · 17%
- Copy into the framebuffer 2,299 · 6%
- HDMI scaler 2,034 · 5% · CRT output 970 · 2%
- Framework, ARM, bus, audio, OSD 5,185 · 13%
- Accelerator queue and bridges 1,146 · 3%

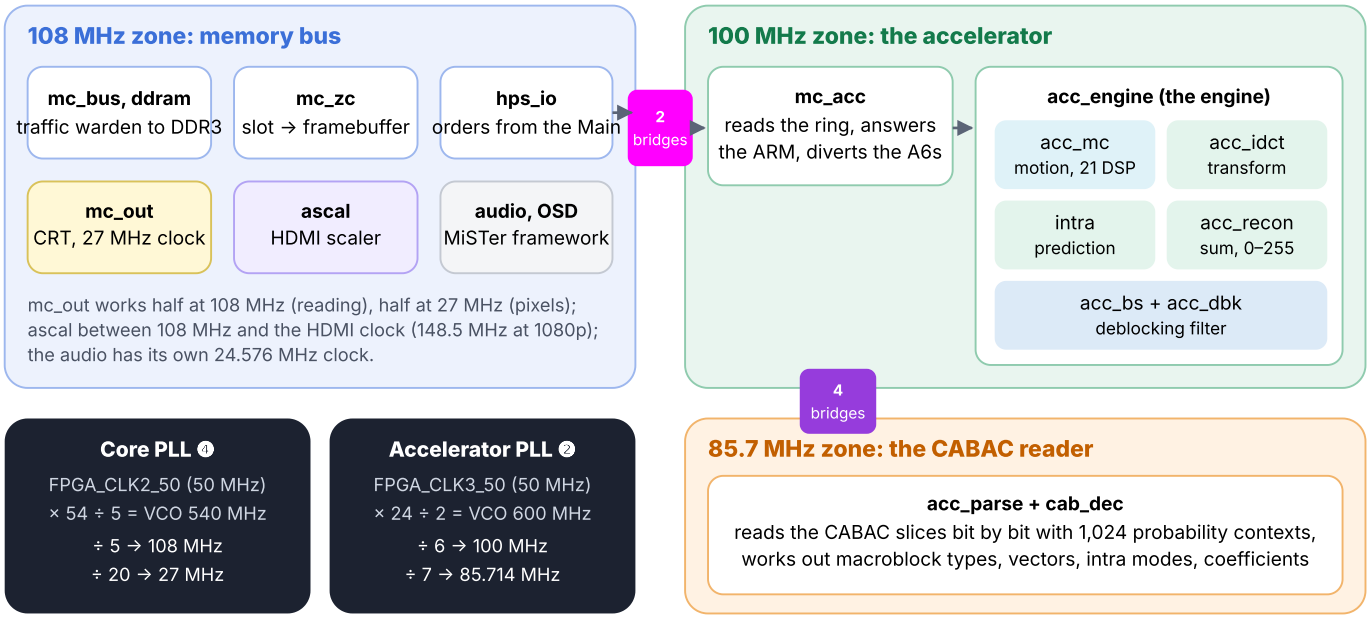
What it means. Engine and CABAC reader together take 70% of the logic: they are the heart of versions 4.0 and 4.5. Since few ALMs are left, any new function in the FPGA must be designed to save logic, or something else must go. Memories and multipliers still have room.

Other numbers: 4,187 LABs used of 4,191; 56,718 logic functions (ALUT); 1,399,503 memory bits of 5,662,720.

Source: Quartus 17.0.2 fitter report, seed 12

5 The clocks and blocks of the core

A digital circuit moves forward at each clock beat, like an orchestra following the conductor. The core (the same in 4.0 and 4.5) has zones that beat at different rates; data move from one zone to another only through small queues called bridges.



Why three zones

With the chip 94% full the wires are long and a signal takes longer to arrive. The engine did not meet timing at 108 or 104 MHz, but did at 100 MHz; acc_parse has long chains of calculation (each bit depends on the previous one) and has its own slower clock, 85.7 MHz. The memory bus and the video outputs stay at 108 MHz as in 3.6.

The **bridges** (`mc_afifo`) are queues whose counters use Gray code: from one number to the next only one bit changes, so a reader on another clock never sees a half-changed value. Two bridges link bus and accelerator, four link engine and acc_parse.

The other two generators

HDMI PLL ❶ (from FPGA_CLK1_50): the Main reprograms it at each resolution; Quartus checks timing on the fastest case, 148.5 MHz for 1080p. **Audio PLL ❸** (from FPGA_CLK1_50, fractional PLL, VCO 417.792 MHz ÷ 17): 24.576 MHz for I2S, S/PDIF and analogue output.

Fixed position. The accelerator PLL is locked at `FRACTIONALPLL_X0_Y56` in the file `menu.qsf` and takes its 50 MHz from pin FPGA_CLK3_50. Placed next to the HDMI PLL, it disturbed its reprogramming and HDMI went out of range at 720p.

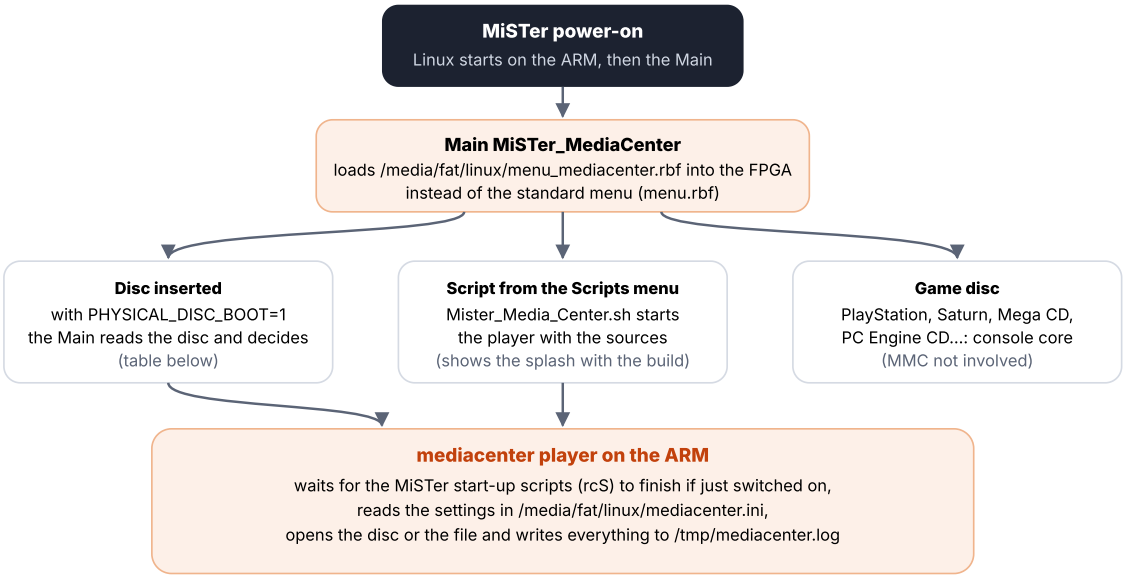
Seed 12 timing (worst slack in nanoseconds, positive = met)

Clock	Period	Slow 100 °C	Slow −40 °C	Fast 100 °C	Fast −40 °C	Worst hold
Bus 108 MHz	9.259	+0.273	+0.220	+4.128	+4.979	+0.075
Engine 100 MHz	10.000	+0.318	+0.319	+4.548	+5.440	+0.025
acc_parse 85.7 MHz	11.667	+0.347	+0.608	+5.477	+6.442	+0.122
CRT video 27 MHz	37.037	+0.577	+0.707	+4.612	+5.284	+0.104
HDMI 148.5 MHz	6.734	+0.014	+0.221	+2.602	+3.296	+0.120
Audio 24.576 MHz	40.690	+14.430	+13.835	+27.432	+29.077	+0.125

Quartus checks every wire in the slowest and the fastest chip Intel makes, at 100 °C and at −40 °C. Setup: the signal arrives before the next clock beat. Hold: it does not change too soon after the beat. Nine seeds were tried on the final code: only seed 12 meets timing in all four conditions.

6 Journey 1: from the source to the player

Before a picture appears, the MiSTer must load the right core, work out what is in the drive or which file was chosen, and start the `mediacenter` program on the ARM.



What the Main decides for a disc

In the drive	How it recognises it	Who opens it
DVD-Video	has `VIDEO_TS/VIDEO_TS.IFO`	mediacenter /dev/sr0
Video CD, Super Video CD	sector 150 starts with VIDEO_CD, SUPERVCD or HQ-VCD	mediacenter
Data DVD with video or music	media files on the disc	mediacenter (file list)
Audio CD	audio tracks only	**mediacenter** (new), or MiSTer Hi-Fi (page 15)
Games	console signature	console core

The sources in the player's list

Entry	Folder
Disc in the drive	the first `/dev/srN`
SD: Video, SD: Music	`/media/fat/Video` , `/media/fat/Music`
SD: whole card	`/media/fat`
USB 0...7	`/media/usbN` , without duplicates
Network (CIFS)	`/media/fat/cifs`
Jellyfin	home server, pages 12–13
CRT Adjust	size and position on the tube TV

New in 4.5: Jellyfin comes over the network (pages 12–13); audio CDs and tracker modules go through a WAV file built on the fly (page 15).

How the data are read

File (MP4, MKV, AVI, MPEG, Ogg, ASF, MP3, FLAC...)
FFmpeg opens the container and splits it into video, audio and subtitle packets, each with its own time.

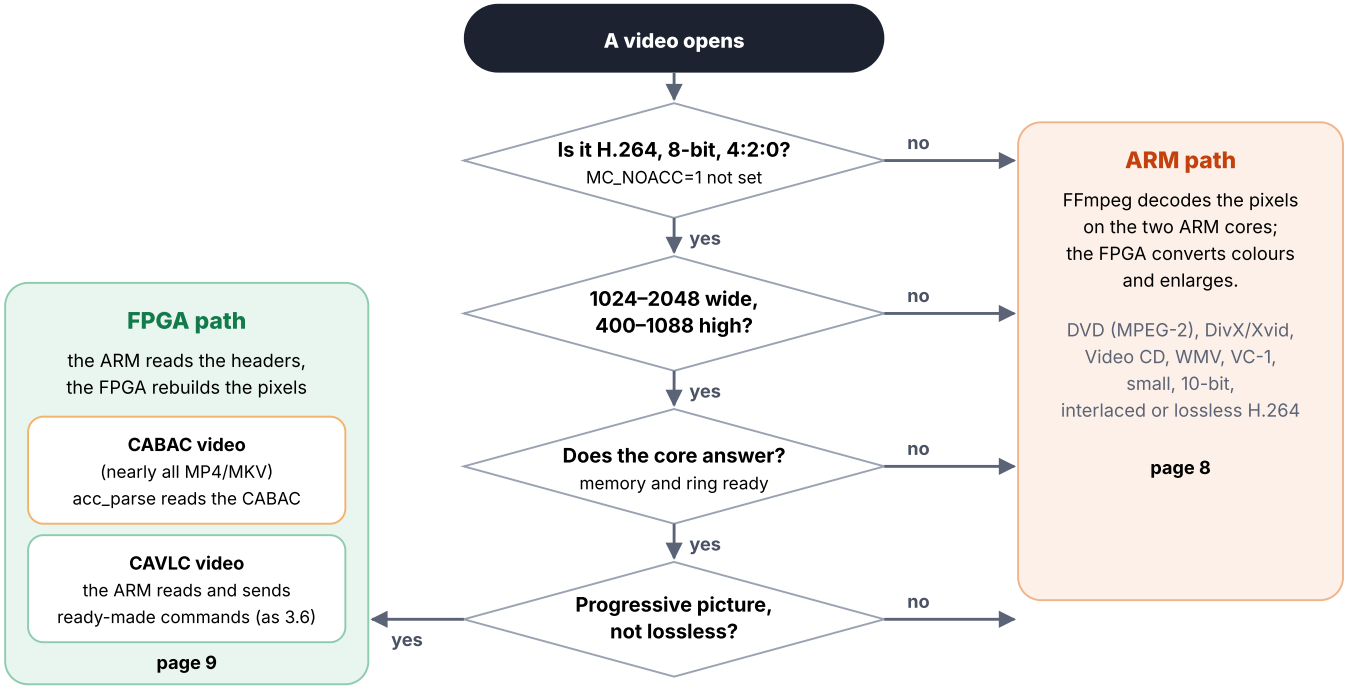
DVD-Video
libdvdnav follows menus, titles and chapters and passes 2048-byte blocks; `dvd.c` splits them into MPEG-2 video, audio and subtitles. Protected discs use libdvdcss from `/media/fat/dvdcss/` .

Video CD and Super Video CD
The player reads the sectors of the video tracks straight from the disc and passes the MPEG-1 or MPEG-2 stream to FFmpeg, as for files.

For DVDs the player uses a reading thread on one core, so a slow disc read does not stop the picture. All decoders come from FFmpeg 6.1.2, built with only the formats needed.

7 Journey 2: who decodes what

For every video the player chooses a path. The ARM can decode everything, but is slow with high-definition H.264; the FPGA is very fast but only does 8-bit progressive H.264. The choice is made when the video opens, also for videos from Jellyfin.



The last question is asked by the modified FFmpeg, picture by picture: if an interlaced, lossless, above-8-bit or 4:2:2 picture arrives, the accelerator switches off and the player reopens the video on the ARM from the same point. The lossless test video (Sega intro) moves to the ARM this way and shows correctly.

CABAC on the FPGA: when yes and when no

Situation	Who reads the CABAC
4.0 core (bit 22 of word 20 set)	**acc_parse in the FPGA**
`MC_NOCABAC=1` in mediacenter.ini	the ARM, as in 3.6
3.6 core or older	the ARM, as in 3.6
acc_parse finds a damaged slice	the ARM, until the next IDR
CAVLC video	no CABAC: the ARM reads the data

What it means, in numbers

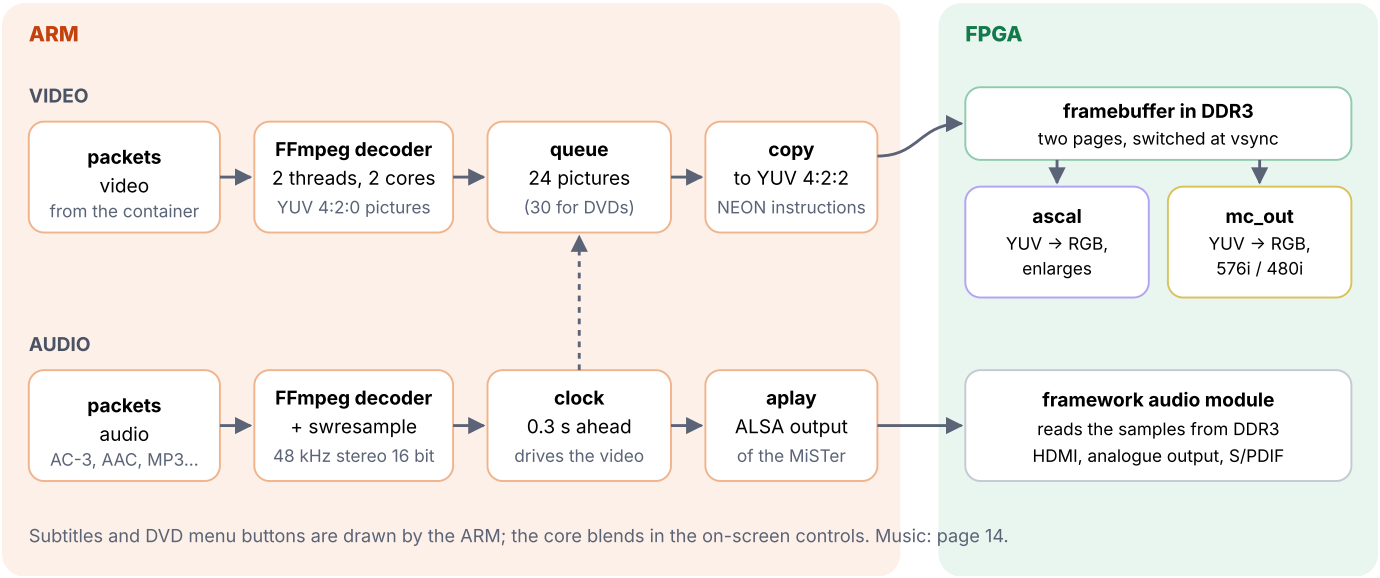
Video (H.264)	ARM 3.6	ARM 4.0
HTC, 1920×1080, 30 fps	26.7 ms	**0.4 ms**
One Piece 940, 1920×1080	12.3 ms	**0.3 ms**
Inuyasha, 1440×810	5.6 ms	**0.4 ms**
Tears of Steel, 1728×720	3.8 ms	**0.3 ms**
One Piece 938, 1280×720	1.7 ms	**0.3 ms**

Average ARM work per picture, measured on the MiSTer on 9 November 2026. The time available is 41.7 ms at 24 fps and 33.4 ms at 30 fps.

Why not put everything in the FPGA? The chip is already 94% full: an MPEG-2 or DivX decoder would not fit next to H.264, and it is not needed. A DVD (720×576) or a DivX is light: the ARM decodes it easily (in the 9 Nov tests: DivX with 1 late picture in 54 s, original DVD with 4 late out of 4,247). The FPGA is needed where the ARM cannot cope, that is high-definition H.264, with up to 8,160 macroblocks per picture against 1,620 for a DVD.

8 Journey 3: the ARM path and the sound

For DVD, DivX, Video CD and the other formats the ARM does the heavy part, the decoding; the FPGA is left with colour conversion and enlarging for the TV. Sound always takes this path, for every video.



Subtitles and DVD menu buttons are drawn by the ARM; the core blends in the on-screen controls. Music: page 14.

The steps, in plain words

1. **Decoding.** FFmpeg turns each packet into a YUV picture (brightness and two colour components). It uses two threads, one per ARM core.
2. **Queue.** Pictures wait their turn in a queue of 24 (30 for DVDs): the reserve for heavy scenes. At start the player waits until it is full, 2 seconds at most.
3. **Copy.** When it is time, the ARM copies the picture into the framebuffer as YUV 4:2:2, half the bytes of RGB. If the picture is wider than the screen it narrows it during the copy with NEON instructions.
4. **Colours and size.** The core converts YUV to RGB: the scaler `ascal` for HDMI, `mc_out` for the CRT. The scaler can only enlarge, never shrink.

Who keeps time

The sound is the clock. The player sends the samples to `aplay` about 0.3 s ahead and always knows which moment is being heard. Each picture is shown when its time comes: if it is late the player skips it, and the log counts it as *not shown*.

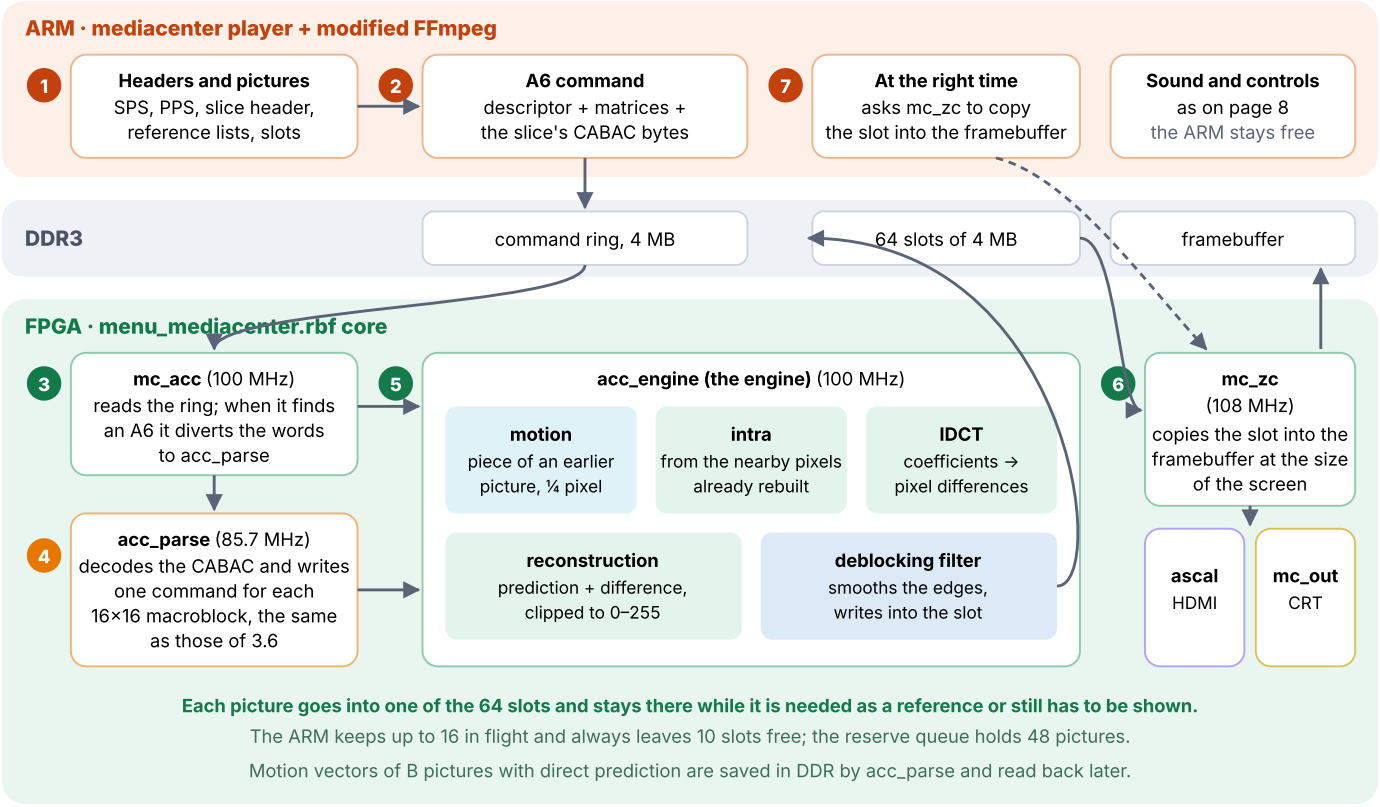
Video CD and DVD

Video CD (MPEG-1 352×288) and DVD (MPEG-2 720×576) are small and the ARM decodes them easily. For DVDs the player uses a framebuffer the same size as the video, so on the CRT each DVD line becomes one TV line.

> The core also does **zero-copy** for the ARM: with a 1080p decoded by the ARM, the `mc_zc` block can copy the picture into the framebuffer itself. `MC_NOZC=1` turns it off.

9 Journey 4: the FPGA path

For a high-definition H.264 video the ARM is only the postman: it reads the headers, prepares a small descriptor and passes the compressed bytes to the FPGA. Everything else, down to the pixels on screen, is done by the core.



Why it is so fast

CABAC is the most compressed way H.264 writes its data: each bit depends on the ones before, so it must be read one after another. For a processor it is the heaviest part of decoding. In the FPGA it is done by `cab_dec`, a dedicated circuit with the probability tables inside it, while the ARM stays free.

Before, the ARM wrote about 1 MB of commands for each 1080p picture. Now it writes only the compressed bytes, 10–20 KB: the DDR3 memory is less crowded too and the engine waits less (One Piece 1080p from 14.7 to 9.0 ms per picture).

Engine numbers

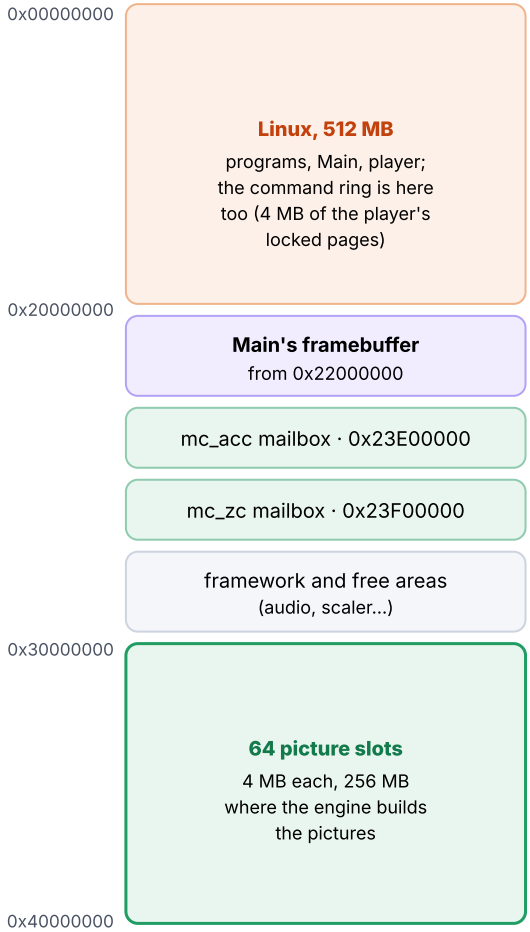
Formats	Progressive 4:2:0 8-bit H.264, Main and High profiles (8×8 transform, matrices)
Largest picture	8,192 macroblocks (1080p has 8,160)
Slot	64 of 4 MB from `0x30000000`, Y lines of 2048 bytes
Resizing	1080p → 1280 or 960 dots; 810p → 960; other sizes resampled
Errors	0 pictures lost by the FPGA and 0 CABAC errors in all 9 Nov tests

A 1080p picture in numbers (One Piece 940, 24 fps, 9 Nov test)

Step	3.6	4.0	What it means
ARM work per picture	12.3 ms	**0.3 ms**	the processor is almost idle: time is left for sound and menus
Data written for the FPGA	1,259 KB	**15 KB**	about 85 times less memory traffic
FPGA pace	40.3 ms	40.5 ms	one picture about every 41.7 ms, as the video asks
Engine time without commands	13.1 ms	22.3 ms	the engine finishes early and waits: there is margin
DDR3 memory wait	14.7 ms	**9.0 ms**	fewer queues on the shared memory

10 The shared memory and the messages

The DE10-Nano has 1 GB of DDR3. Linux uses the first half; the second belongs to the FPGA. ARM and core exchange everything through areas at fixed addresses, like letterboxes.



How to read it. Addresses are physical, the same for ARM and FPGA. The player reaches them through `/dev/mem`; the core through its direct DDR3 access (f2sdram ports). The ring lives in the player's memory, which Linux scatters in 4 KB pages: a page table tells the core where each one is.

The mc_zc mailbox (signature `MCZC`) receives the copy orders: which slot, which framebuffer page, at what size. From +0xC0000 it holds the page tables of the 64 slots.

The command ring

A 4 MB circular queue, like a race track: the player writes at the end and moves its pointer; mc_acc reads from the start and says how many bytes it has read. At the end it starts over from the beginning. The player writes without going through the cache and, before moving the pointer, has the L2 cache (PL310) buffer flushed: the FPGA reads the DDR and does not see the ARM caches.

The mc_acc mailbox words

The player writes its words: the signature `MCAC`, the session number, where the ring is and how far it has written. The core answers from +64 bytes with the signature `ACOK`, the bytes read, the pictures finished, the timings and word 20, which says what it can do:

Bit	Meaning	Bit	Meaning
0	bad command to the engine	18	deblocking filter in the core
1	memory not answering	19	the core blends menu and video
2	CABAC slice with errors	20	the core resizes (720p...)
16	core 3.0 or newer	21	accelerator at 100 MHz
		22	**acc_parse present (A6)**

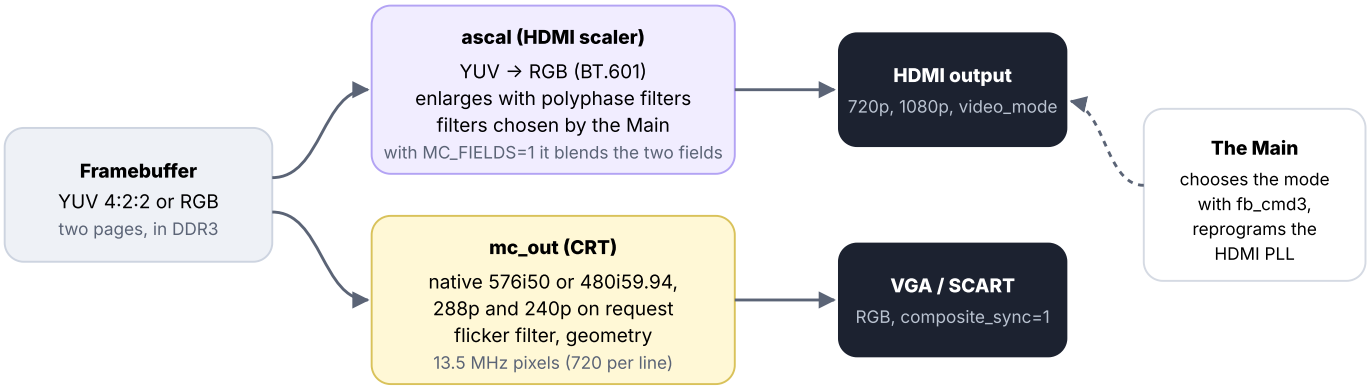
The A6 command

32-bit words	Contents
1	A6 and the number of words that follow
5	descriptor: slot of the picture and of the co-located one, size, first macroblock, slice type, QP, lists, filter, offsets
0 or 96	only B with temporal direct: distances and reference map
variable	only with implicit weights: the weights for each reference pair
24 (+32)	4×4 (and 8×8) quantisation matrices
1 + n	number of bytes, then the slice bytes after CABAC alignment, 4 per word

All engine commands (A0 fill, A5 filter, macroblocks...) are described in `accel/doc/comandi.md`.

11 HDMI and CRT outputs, the logs

The same framebuffer goes to two TVs at once: HDMI through the MiSTer scaler, the CRT through the 15 kHz output written for MediaCenter.



Settings in MiSTer.ini

```
[MiSTer]
vga_scaler=0
forced_scandoubler=0
composite_sync=1      ; with RGB SCART

[menu]
main=MiSTer_MediaCenter
PHYSICAL_DISC_BOOT=1

[A0CD-*]
main=MiSTer_Physical-CD  ; game discs
```

The file `MMC_MiSTer_ini.txt` in the zip holds these lines to copy. Game discs need the MiSTer Physical Disc Main (Anime0t4ku), downloaded separately.

Useful settings in mediacenter.ini

`MC_FIELDS=1`	keeps the fields of interlaced video (recommended with the CRT)
`MC_CRT_MODE`	0 auto, 1 576i, 2 288p, 3 480i, 4 240p
`MC_CRT_HSIZE…VPOS`	size and position on the CRT (also from CRT Adjust)
`MC_AV_OFFSET`	shifts sound against picture
`MC_NOACC=1`	turns the accelerator off (all on the ARM)
`MC_NOCABAC=1`	CABAC on the ARM as in 3.6
`MC_JELLYFIN=`	Jellyfin server address (empty: search for it; 0: hide the entry)
`MC_AUDIO_W=720`	full-width audio screen
`MC_DIAG=0`	no diagnostic lines in the log

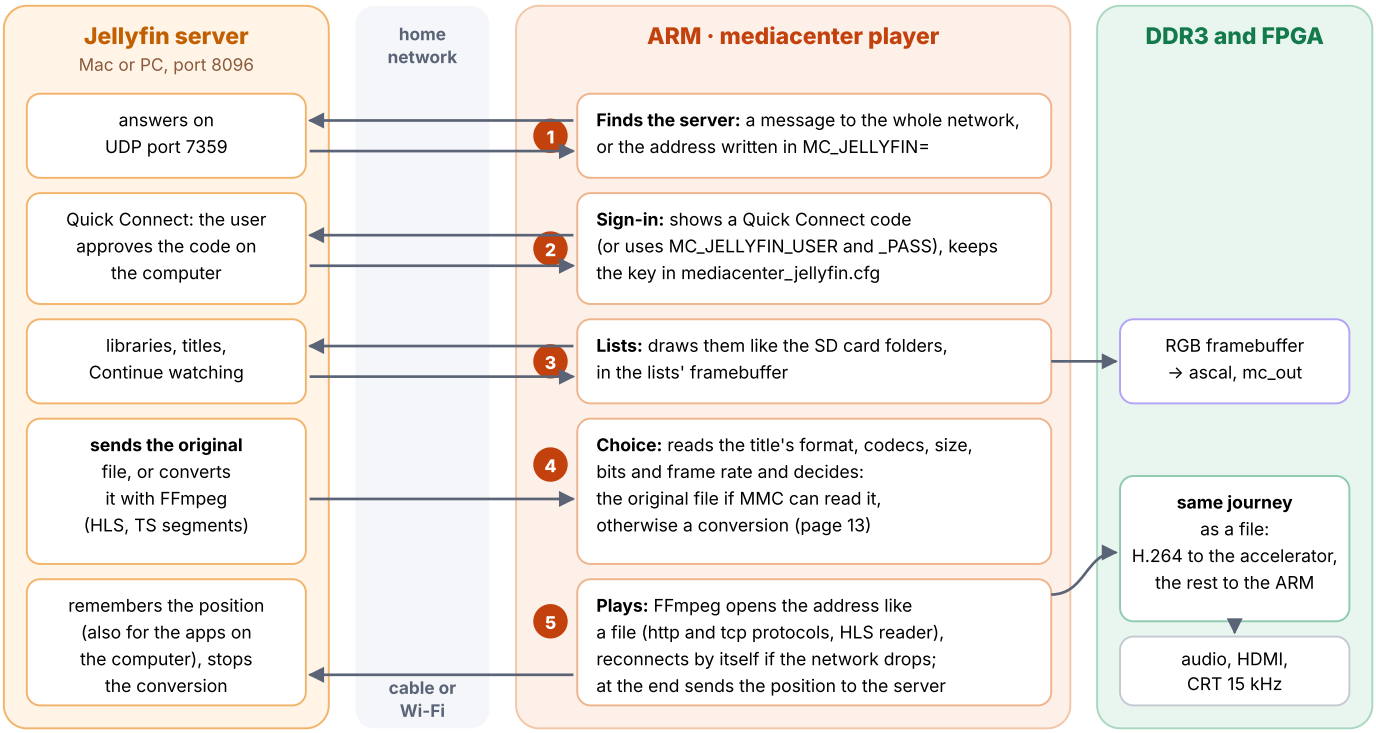
The three logs to read when something goes wrong

File	Who writes it	What it holds
`/tmp/mediacenter.log`	the player (rewritten for each video)	video format, path chosen (*diag acceleratore: acceso … CABAC nell'FPGA (accelerator on, CABAC in the FPGA)*; with Jellyfin *file originale (original file)* or *convertito dal server (converted by the server)*; the log is in Italian), every 10 s: pictures shown, not shown, late, lost by the FPGA, KB per picture, ARM and FPGA times, memory wait
`/tmp/mediacenter_main.log`	the Main	which menu core was loaded and which Main runs the menu
`/tmp/mediacenter_crt`	the Main	CRT video mode, picture size and position

A good line, for comparison (One Piece 1080p, 9 Nov): accelerator on, CABAC in the FPGA; 0.1% of pictures not shown, 0 late, 0 lost by the FPGA, about 15 KB per picture, ARM 0.3 ms.

12 Journey 5: Jellyfin, from the network to the player

Jellyfin is a free program that runs on a home computer and keeps films, series and music in order. MMC talks to it with the HTTP requests of its public API, the same as the official apps, with code written for MMC (`jellyfin.c`). It all happens on the ARM: for the FPGA a film from Jellyfin is a video like any other.



The requests, in order

Step	Request to Jellyfin
1 search	"who is JellyfinServer?" over UDP port 7359, up to 3 times; the server answers with its address and name
2 sign-in	`/System/Info/Public`, `/QuickConnect/Initiate`, `/QuickConnect/Connect`, `/Users/AuthenticateWithQuickConnect` (or `/Users/AuthenticateByName`)
3 lists	`/UserViews` (libraries), `/UserItems/Resume` (Continue watching), `/Items?ParentId=` (up to 2,000 entries)
4 choice	`/Items/{id}` : the *MediaSources* with the video and audio streams
5 stream	`/Videos/{id}/stream?static=true` or `/Videos/{id}/master.m3u8` ; music `/Audio/{id}/stream`
reports	`/Sessions/Playing` at the start, `/Sessions/Playing/Stopped` at the end, then `DELETE /Videos/ActiveEncodings` if there was a conversion

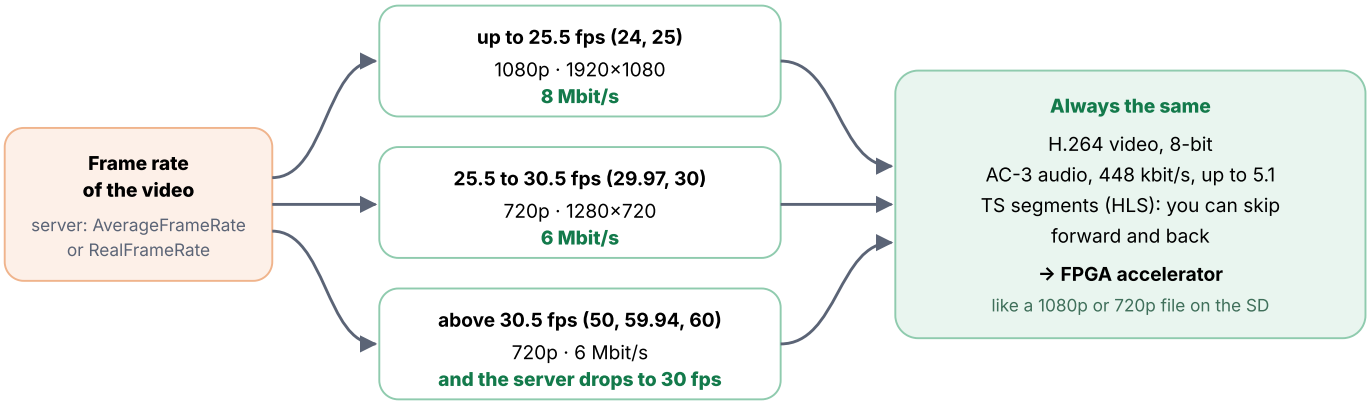
What arrives as the original

Video	8-bit H.264 up to 1920×1088, MPEG-1 and 2, MPEG-4 (DivX, Xvid), H.263, MJPEG, WMV, VC-1
Audio	MP3, AC-3, E-AC-3, DTS, AAC, Vorbis, FLAC, ALAC, Opus, WMA, PCM
Containers	MKV, WebM, MP4, MOV, AVI, TS, MPEG, VOB, ASF, Ogg, WAV and the audio files
Converted	HEVC, AV1, VP9, 10-bit H.264, above 1920×1088, discs (ISO images, DVD or Blu-ray folders), audio such as TrueHD
Music	original; what MMC cannot read arrives as FLAC

> **Why the original file matters:** the same H.264 arrives that would be on the SD card, so the FPGA accelerator, audio tracks and subtitles, the CRT and the audio screen all work, with no loss of quality.

13 Jellyfin: conversion, resume and numbers

When MMC cannot read a video (HEVC, AV1, 10-bit, 4K...), it asks the server to convert it to 8-bit H.264: so it reaches the FPGA accelerator again. The request is tuned to the work the accelerator handles with margin.



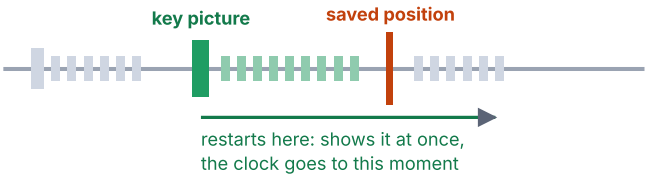
Why these values

The accelerator's time grows with the **bytes of each picture**: more compressed data means more work for the CABAC reader and more memory accesses. In the first test at 12 Mbit/s each picture weighed about 49 KB and the FPGA took 35–43 ms, more than the 33.4 ms a 30 fps video allows: the video stuttered.

Request	KB per picture	Time available
12 Mbit/s at 30 fps (first test)	about 50	33.4 ms: not enough
8 Mbit/s at 24–25 fps	40–42	40–41.7 ms
6 Mbit/s at 30 fps	25	33.4 ms

KB per picture = bits per second ÷ 8 ÷ frames per second (averages; key pictures weigh more). A 60 fps video would give the FPGA only 16.7 ms per picture: that is why the server brings it to 30 (`MaxFramerate=30`).

Resuming where you stopped



Before: straight jump, 440 pictures dropped and gaps in the sound.

If Jellyfin remembers a position past 30 seconds, MMC asks *Riprendi da... (resume from)* or *Dall'inizio (from the start)*. To resume, the player opens the video and restarts from the key picture just before the position: it decodes and shows it at once, sets the clock to that moment and fills the picture reserve before starting. So no picture arrives late. The position comes from Jellyfin, so a film left halfway on the computer works too.

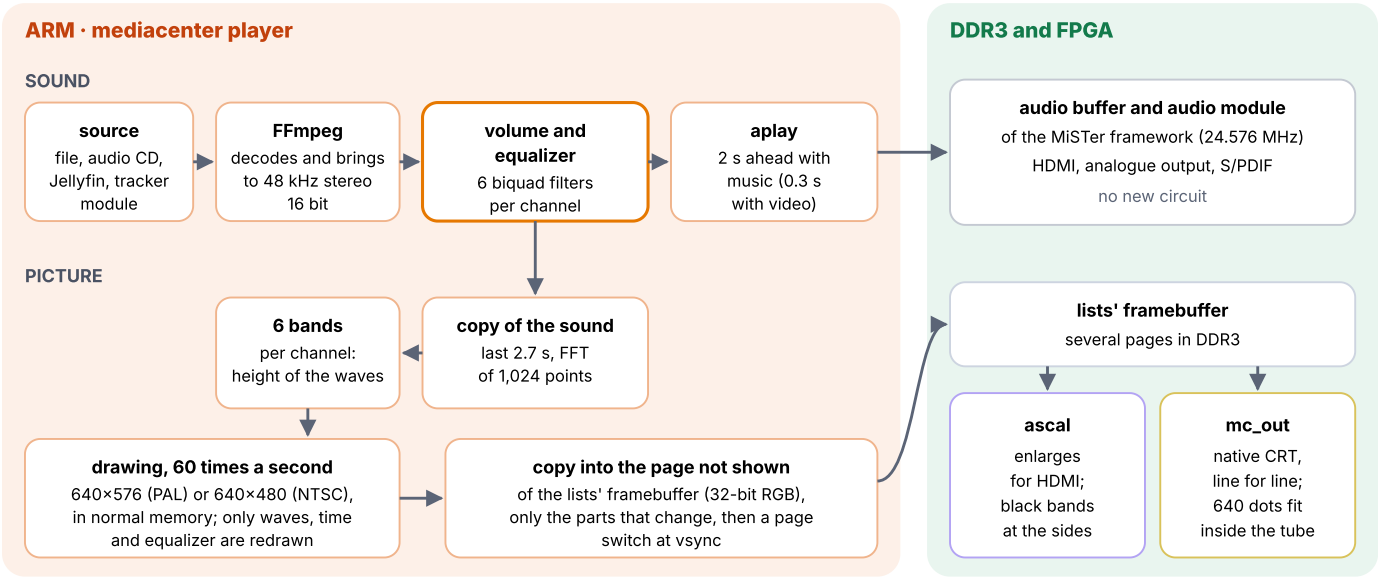
Tests on the MiSTer, 11 and 12 November 2026 (official server on a Mac mini M2 Pro)

Video from Jellyfin	Path	Result
Inuyasha, H.264 1440×810 MKV, 1,410 s	original, accelerator	0.0% of pictures not shown, none lost by the FPGA
Tears of Steel, H.264 1728×720, 734 s	original, accelerator	0.0% of pictures not shown
HEVC at 30 fps	converted to 720p, 6 Mbit/s	0.8% of pictures not shown
HEVC at 59.94 fps	converted to 720p at 30 fps	0.3% of pictures not shown
Resume where you stopped	original	48 pictures ready in 0.24 s, none skipped, clean sound
FLAC and MP3 music	original	audio screen, no problems

When conversion is heavy for the server. Converting 4K HEVC is a big job even for a computer: in the tests the Mac mini M2 Pro struggled with 4K. If the server cannot keep up, the video stutters even though the MiSTer is fine. For hardware acceleration on the server see the official Jellyfin documentation.

14 The audio screen and the equalizer

When an audio file, a CD or a track from Jellyfin plays, the ARM draws a live screen: name, folder and format, two waves that follow the music, the time, the controls and a 6-band equalizer. The FPGA shows it the way it shows the lists.



The equalizer

Band	Filter
60 Hz	low shelf (raises or lowers all the bass)
170, 470, 1.3k, 3.6k Hz	peak (only around that frequency)
10 kHz	high shelf (all the treble)
Vol	0 to −5 dB, for songs recorded loud that crackle on tube TV speakers

Each band goes from −12 to +12 dB. The calculation costs the ARM about 1–2% of one core; with all bands at 0 the sound stays identical. Settings stay for the following tracks and are saved in `/media/fat/linux/mediacenter_eq.cfg`. The equalizer works only with the audio screen, not on videos.

The buttons

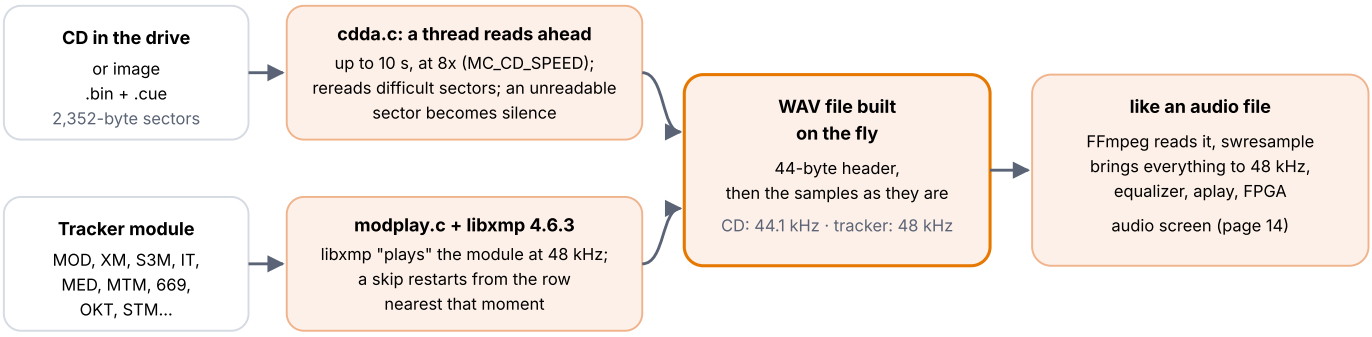
Y	opens and closes the equalizer; it closes by itself after 10 s without buttons
left, right	with the equalizer open: chooses the band or Vol
up, down	raises or lowers by 1 dB
A	screen saver on or off (stays for the following tracks)
B	leaves the track and stops the sound at once: throws away what was sent ahead and closes aplay (the same with next or previous track)
pause	goes back to the moment being heard, so the sound stops at once

Settings: `MC_AUDIO_W=720` brings back the full-width screen; `MC_AUDIO_SIMPLE=1` brings back the old screen (title and format only). With the CRT at 288p or 240p two lines of the drawing become one (averaged).

Why 640 dots: on the CRT the 720-dot screen went partly outside the tube; at 640 it fits entirely inside. HDMI and CRT read the same framebuffer, so HDMI also shows two thin black bands at the sides.

15 Audio CDs, new formats, Main, script and picture

Audio CDs and tracker modules are not files FFmpeg can open by itself. The player turns them into a "fake" WAV file, built on the fly in memory: so pause, skips, fast forward, equalizer and audio screen work as for an MP3.



Opus files need no tricks: FFmpeg now includes the Opus decoder and opens them like the other audio files.

The audio CD, from the disc to the screen

1. The Main sees a disc with only audio tracks. If `MiSTer.ini` has `AUDIOCD=MEDIACENTER`, or there is no `AUDIOCD=` and MMC is installed, it starts the player with the name of the CD drive.
2. The player reads the disc's table of contents and the **CD-Text** (title and artist of each track); for an image it reads the TITLE and PERFORMER lines of the `.cue`.
3. All tracks become one stream, with no pause between tracks: the tracks are the chapters, left and right change track.
4. The audio screen shows the track's number, time and name.

With `AUDIOCD=MISTERHIFI` the CD goes to MiSTer Hi-Fi as before.

The Main in line with the official one

The MMC Main is the official MiSTer Main plus the MMC parts. In 4.5 it was merged with the official Main of 11 November 2026 (commit `04f91c6`, merge `0c6f8fe7`): it brings the new controller mapping **joymap v4**, force feedback requested by cores and the Minimig/CD32, Atari ST, Apple IIgs and other core updates released after 16 October.

The MMC parts stay as before: disc detection, loading of `menu_mediacenter.rbf`, the `fb_cmd3` command for the colour format, CRT output. The core does not change, so neither does the dialogue between Main and FPGA.

The script is called Mister_Media_Center.sh

It used to be called `mediacenter.sh`: only the name changes, the contents are the same (MD5 `395f1811…` in both versions). The name has no spaces because the Main starts scripts with `cd $(dirname %s)` without quotes: a space would break the path. When updating from 4.0, delete the old `/media/fat/Scripts/mediacenter.sh`, otherwise two entries appear.

The start-up picture

The picture *Ver. 4.5 H264 Jellyfin Client 1080 P* (`docs/mmc_splash.png`) is converted by `make_splash.py` into `splash.h`: 640×480 dots, 256 colours, run-length coded (RLE). The player draws it in the lists' framebuffer for 3 seconds, with the build number bottom right, **only when started from Scripts**. A disc started by the Main goes straight to the film or the CD. `MC_SPLASH=0` turns it off; any button skips it.

16 Rebuilding from scratch (1/2): ingredients and core

MiSTer MediaCenter is made of three programs and a circuit. All come from the same repository, at release v4.5. With this page and the next one every file in the zip can be made again.

File in the zip	Folder on the SD	Where it comes from	MD5 in release 4.5
`menu_mediacenter.rbf`	/media/fat/linux/	Quartus, folder `fpga/menu`	`7cb8175f995d5b72a49c2ef032fbc963`
`mediacenter`	/media/fat/linux/	`make mediacenter`	`c46efb131ba5947eaced0a9c08c89de6`
`MiSTer_MediaCenter`	/media/fat/	`make` (the Main)	`eb6e97a3a906a8ce4d172bb913a27d69`
`Mister_Media_Center.sh`	/media/fat/Scripts/	`Scripts/` in the repository	`395f18116ad0a3156ccc3500e551e2d2`
`libdvdcss.so.2`	/media/fat/dvdcss/	`dvdcss/` in the repository	`7a140afb27296b1a7417358bbcc44cfd`
`MMC_MiSTer_ini.txt`	/media/fat/	`docs/` in the repository	lines to copy

Step 0 · The code

```
git clone https://github.com/bonna79/MiSTer_MediaCenter.git
cd MiSTer_MediaCenter
git checkout 307a7c5      # release v4.5: everything in the zip
```

The core circuit is identical to commit `506375b`, the one tested on the MiSTer for 4.0: from there to 4.5 the `fpga/` and `accel/` folders have not changed. What changed is the ARM programs, the script, the start-up picture and the documentation.

Step 1 · The FPGA core, identical bit for bit

Quartus is **deterministic**: with the same sources, the same version, the same seed and the same date written into the core it produces the same file. If just one of these changes, the whole placement changes, and with the chip 94% full timing may no longer be met.

Version
Quartus Prime **17.0.2** Build 602 Lite Edition (free)

Seed
`set_global_assignment -name SEED 12` into `fpga/menu/menu.qsf`

Fixed date
`fpga/menu/build_date.txt` = `261027`, written into the core by `sys/build_id.tcl`

```
docker run --rm -v "$PWD:/build" -w /build/fpga/menu \
  theypsilon/quartus-lite-c5:17.0.2 \
  quartus_sh --flow compile menu.qpf
# docker image: sha256:42e6c06da3af486f9cfea74acb5c1af9b63a77d0f16e3bf093215216e209b7ad
md5sum fpga/menu/output_files/menu.rbf   # must give 7cb8175f995d5b72a49c2ef032fbc963
```

The identical result is proven with this Docker image (and on GitHub). With a Quartus 17.0.2 Lite installed by hand open `fpga/menu/menu.qpf`, compile and check the MD5. Or on GitHub: *Actions → Build MediaCenter menu core → Run workflow* (about 35 minutes, file `.github/workflows/fpga.yml`). Rename `menu.rbf` into `menu_mediacenter.rbf`.

> **If the MD5 differs**, check in order: Quartus version, `SEED 12`, date `261027`, PLL position `FRACTIONALPLL_X0_Y56_N0` into `menu.qsf`. If even one line of Verilog changes, seed 12 no longer holds: other seeds must be tried until one meets all timing in the four models (slow and fast, 100 °C and −40 °C) with `quartus_sta`.

The file `fpga/menu/sys/emu_ports.vh` may show as modified in git because of line endings (LF/CR+LF): Quartus reads it the same and the core does not change. The full placement, cell by cell, is in the attachment `seme12_piazzamento.zip` of the 4.0 documentation (120,318 cells, timing reports, menu.qsf).

17 Rebuilding from scratch (2/2): player, Main, package

The ARM programs are compiled on a Linux PC (GitHub uses Ubuntu 22.04) with ARM's cross compiler. With the same tools the result is equivalent, though not always identical byte for byte.

Step 2 · The compiler

```
# basic tools (Ubuntu): git make patch curl wget bzip2 xz-utils zip pkg-config
# ARM GNU Toolchain 10.2-2020.11, arm-none-linux-gnueabihf, for x86_64 PCs (exact address from build.yml)
wget -qO- "https://developer.arm.com/-/media/Files/downloads/gnu-a/10.2-2020.11/binrel/gcc-arm-10.2-2020.11-\
x86_64-arm-none-linux-gnueabihf.tar.xz?revision=d0b90559-3960-4e4b-9297-7ddbc3e52783&rev=d0b9055939604e4b\
92977ddbc3e52783&hash=0074C1529DE90C98726B80ED3EE0776C" | tar -xJ -C /opt
export PATH=/opt/gcc-arm-10.2-2020.11-x86_64-arm-none-linux-gnueabihf/bin:$PATH
```

Step 3 · The player's libraries

```
# FFmpeg 6.1.2: the script applies tools/mediacenter/ffmpeg-mediacenter.patch
git clone --depth 1 -b n6.1.2 https://github.com/FFmpeg/FFmpeg.git ffmpeg-src
tools/mediacenter/build_ffmpeg.sh ffmpeg-src "$PWD/ffmpeg-arm"
# libdvdread 6.1.3 and libdvdnav 6.1.1, for DVDs
curl -fsSL https://download.videolan.org/pub/videolan/libdvdread/6.1.3/libdvdread-6.1.3.tar.bz2 | tar -xj
curl -fsSL https://download.videolan.org/pub/videolan/libdvdnav/6.1.1/libdvdnav-6.1.1.tar.bz2 | tar -xj
tools/mediacenter/build_dvdnav.sh libdvdread-6.1.3 libdvdnav-6.1.1 "$PWD/dvdnav-arm"
# libxmp 4.6.3 (MIT licence), for tracker modules
curl -fsSLO http://archive.ubuntu.com/ubuntu/pool/universe/libx/libxmp/libxmp_4.6.3.orig.tar.gz
# sha256 b189a2ff3f3eef0008512e0fb27c2cdc27480bc1066b82590a84d02548fab96d
tar -xzf libxmp_4.6.3.orig.tar.gz
tools/mediacenter/build_libxmp.sh libxmp-4.6.3 "$PWD/xmp-arm"
```

FFmpeg is built static for Cortex-A9 with NEON and only the decoders needed: video MPEG-1/2/4, DivX, H.263, H.264, MJPEG, WMV, VC-1; audio MP1/2/3, AC-3, E-AC-3, DTS, AAC, Vorbis, FLAC, ALAC, **Opus**, WMA, PCM; DVD, SRT, ASS subtitles. For Jellyfin there are networking, the `http` and `tcp` protocols and the `hls` reader. The patch adds the accelerator mode to FFmpeg: instead of rebuilding the pixels, it writes the commands for the FPGA (and the A6 command of the 4.0 core).

Step 4 · The player and the Main

```
make mediacenter FFMPEG="$PWD/ffmpeg-arm" DVDNAV="$PWD/dvdnav-arm" XMP="$PWD/xmp-arm" MC_BUILD=224
#   → bin/mediacenter   (MC_BUILD is the number written on the splash)
make
cp bin/MiSTer bin/MiSTer_MediaCenter
```

Step 5 · The package and installation

```
mkdir -p pkg/linux pkg/Scripts pkg/dvdcss
cp menu_mediacenter.rbf bin/mediacenter pkg/linux/
cp bin/MiSTer_MediaCenter docs/MMC_MiSTer_ini.txt pkg/
cp Scripts/Mister_Media_Center.sh pkg/Scripts/ ; cp dvdcss/libdvdcss.so.2 pkg/dvdcss/
cp LICENSE pkg/linux/mediacenter-LICENSE
(cd pkg && zip -r ../MiSTer_MediaCenter.zip .)
```

Extract the zip to the root of the SD card, copy the lines of `MMC_MiSTer_ini.txt` into `MiSTer.ini` and restart. On GitHub all this is done by `.github/workflows/build.yml`: a commit on main with `[release]` in the message publishes the release, after checking the core's MD5.

Step 6 · The final test

1. The MiSTer menu starts and `mediacenter_main.log` has *MediaCenter menu core*.
2. The script shows the picture with *Build 224*.
3. A 1080p MP4: the log says *diag acceleratore: acceso … CABAC nell'FPGA*, 0 lost.
4. The lossless test video `sega_intro_lossless.mp4` moves to the ARM and the logo shows.
5. An original DVD: menus, audio tracks and subtitles.
6. A Video CD and a DivX: smooth, clean sound.
7. An audio CD and a film from Jellyfin.

18 Glossary

The technical words used in this document, explained simply.

Word	What it means
FPGA	Programmable chip: instead of a program it holds a circuit that can be redrawn. On the MiSTer it is an Intel Cyclone V.
Core (.rbf)	The circuit design loaded into the FPGA.
ARM, HPS	The dual-core 800 MHz Cortex-A9 processor inside the same chip; it runs Linux.
ALM, LAB	The FPGA's logic brick; 10 ALMs make a LAB.
M10K, DSP	Internal 10 kbit memories; fast multipliers.
Clock, MHz	The beat that moves the circuit forward: 100 MHz = 100 million steps a second.
PLL	Clock generator: multiplies an input clock and divides it.
Seed	The number Quartus starts from to place the circuit. Different seeds give different placement and timing.
Setup, hold	The two timing checks: arrive in time and do not change too soon.
Bridge (FIFO)	A queue that passes data between two zones with different clocks.
DDR3	The board's 1 GB memory, shared by ARM and FPGA.
Framebuffer	The memory area holding the picture to show.
H.264	The video format of most MP4 and MKV files.
Macroblock	A 16×16 pixel square: a 1080p picture has 8,160.
CABAC, CAVLC	The two ways H.264 writes compressed data; CABAC compresses more and is slower to read.
Slice	A piece of picture compressed on its own; often the whole picture.

Word	What it means
I, P, B, IDR	Pictures: on their own, from earlier ones, from earlier and later ones; an I from which decoding starts afresh.
IDCT	Inverse transform: from coefficients to pixel differences.
Deblocking	A filter that hides the edges between blocks.
YUV	Brightness plus two colour components; 4:2:2 and 4:2:0 say how much colour is kept.
Slot	One of the 64 spaces of 4 MB where the core builds a picture.
Zero-copy	The core copies the picture into the framebuffer by itself, without the ARM.
MD5	A file's fingerprint: same fingerprint, same file.
Jellyfin	A free program that keeps films, series and music on the home computer and sends them over the network.
Quick Connect	Signing in to Jellyfin with a code approved on the computer, without typing the password.
HLS	Video sent in pieces of a few seconds with an index (`.m3u8`): used for conversions.
Mbit/s	Millions of bits per second: how much compressed data a video has.
Equalizer	Raises or lowers bass, mid and treble; here 6 *biquad* filters per channel.
FFT	A calculation that splits a sound into its frequencies: it moves the waves.
CD-Text	Album and track names written on some audio CDs.
Tracker module	Music made of samples and notes (MOD, XM, S3M, IT).
joymap v4	The official Main's new controller mapping.

19 Source map and thanks

Where everything is in the repository

Folder or file	Contents
`fpga/menu/menu.sv`	the body of the core: PLLs, wiring, menu
`fpga/menu/rtl/`	mc_acc, mc_zc, mc_out, mc_bus, mc_parse, mc_afifo, pll_acc, sdram, ddram
`fpga/menu/sys/`	MiSTer framework: ascal (with YUV), OSD, audio, hps_io
`accel/rtl/`	the engine: acc_engine, acc_mc, acc_idct, acc_dbk, acc_bs, acc_resid, acc_recon
`accel/parse/rtl/`	acc_parse, cab_dec, tables `.svh` and `.hex`
`accel/sim/`, `accel/parse/sim/`	simulations and comparisons with FFmpeg
`accel/doc/comandi.md`	all engine commands
`tools/mediacenter/`	the player: mediacenter.c, dvd.c, accel.c, accel_link.c, zerocopy.c, **jellyfin.c, audio_ui.c, cdda.c, modplay.c**, splash.h, FFmpeg patch, library scripts
`tools/vcd_player/`	the old Video CD player, included in the player (`MC_VCD_OLD=1`)
`*.cpp` in the root	the Main: user_io.cpp loads the core, video.cpp handles the framebuffer and the CRT
`support/physical_disc/`	disc detection, audio CDs to MMC
`Scripts/`	`Mister_Media_Center.sh`
`docs/`	technical sheets, pictures, lines for MiSTer.ini
`.github/workflows/`	`build.yml` (programs and release), `fpga.yml` (core)

Thanks to Umberto Parisi (rmonic79) of the Beceri Videoludici, whose work on MiSTer CRT Adjust made CRT Adjust possible; to Max Riptide for the many tests; to mad4gith for the ideas. MiSTer Physical Disc is by Anime0t4ku. The framework and the Main come from MiSTer by Sorgelig and contributors.

Status on 12 November 2026. Version 4.5 tested on the MiSTer with a Sony Trinitron CRT (RGB SCART) and HDMI together: controllers with joymap v4, original DVD, audio CD, data DVD, Opus, tracker modules, audio screen, Jellyfin with original files (0.0% of pictures not shown), HEVC conversions (0.3–0.8%), resume and music. The 4.0 core tests (page 9) remain valid: the core is the same.

Installing or updating to 4.5

File in MiSTer_MediaCenter.zip	Folder on the SD
`MiSTer_MediaCenter` (Main)	`/media/fat/`
`mediacenter` (ARM player), `menu_mediacenter.rbf` (core)	`/media/fat/linux/`
`Mister_Media_Center.sh` (script)	`/media/fat/Scripts/`
`libdvdcss.so.2`	`/media/fat/dvdcss/`
`MMC_MiSTer_ini.txt` (lines to copy into `MiSTer.ini`, page 11)	`/media/fat/`

From 4.0: extract the zip over the old files, delete `/media/fat/Scripts/mediacenter.sh` and restart. `MiSTer.ini` and `mediacenter.ini` stay as they are. Install the Jellyfin server by following the official documentation (jellyfin.org/docs).

If there are problems open a report on github.com/bonna79/MiSTer_MediaCenter/issues with a photo of the screen and the files `/tmp/mediacenter.log`, `/tmp/mediacenter_main.log` and `/tmp/mediacenter_crt`.